Práctico 06 — FDM, game loop e input

Computación gráfica y ambientes virtuales (0494)

Facultad de Ingeniería
Centro Regional Universitario Córdoba - IUA
Universidad de la Defensa Nacional

2 de octubre de 2026



Roadmap de la clase

Antecedentes

- Un avión armado con primitivas
- Cámara y proyección en la escena
- Captura de entradas de mouse

Hoy

- El **FDM**: qué es y qué pide
- **Game loop** con paso de tiempo fijo
- **Entradas** de teclado

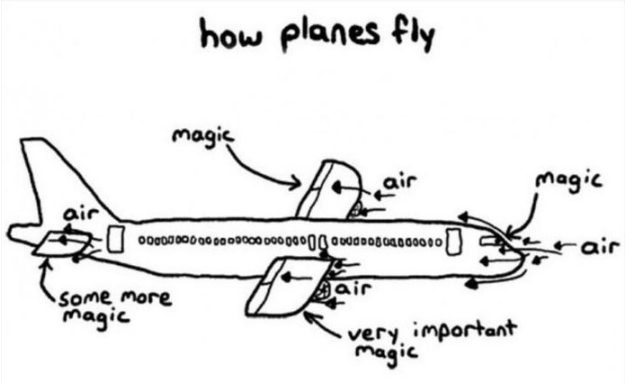
Conectado a

- El proyecto: control de la aeronave
- Terreno y sistema de cámaras

Las cajas negras de hoy

- **El FDM será un caja negra en los detalles** pero hay que entender *qué pide, qué devuelve y por qué hay que integrarlo*. El modelo físico que resuelve seguirá como caja negra: no es contenido de esta materia.

Cómo vuela un avión *101*

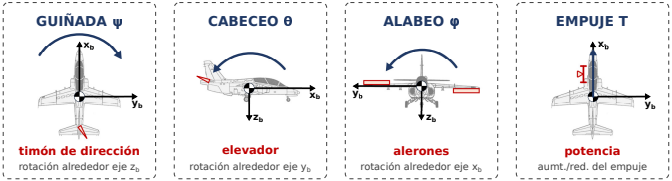


Control básico de una aeronave

Control mediante *tres* grupos de superficies móviles y un control de potencia.

Cada superficie móvil modifica el campo de movimiento del aire de la superficie sustentadora a la que pertenece y cambia la fuerza que esta última genera que a su vez produce un **momento** respecto al CG: como resultado el avión gira.





Ejes cuerpo (convención aeronáutica)
x_b hacia adelante · y_b hacia el ala derecha · z_b hacia abajo (dextrógiro)
Se componen en el orden 3-2-1: $\psi \to \theta \to \varphi$

Posición y orientación: la convención aeronáutica

La **posición** de una aeronave se escribe mediante tres datos: la latitud, la longitud y la altitud. Está última es muy importante porque permite operar con seguridad. La orientación se denomina **actitud** en aeronáutica y se describe mediante los tres ángulos de Euler.

Ángulos en orden 3-2-1

- 3. ψ — guiñada, alrededor de **z**
- 2. θ — cabeceo, alrededor de **y**
- 1. ϕ — alabeo, alrededor de **x**

Posición: tres datos

- **latitud** — posición norte-sur
- **longitud** — posición este-oeste
- **altitud** — dist. vertical sobre el nivel del mar

Sistema de referencia NED

Cuando los cambios de latitud y longitud son pequeños, se reemplazan por un **plano tangente** con origen fijo: sistema dextrógiro **NED** —*north, east, down*—, con la altitud como $-z$. Es el sistema en el que el FDM entrega la posición y sobre el que se definen los tres ángulos.

El orden importa

Las rotaciones no conmutan: ψ y después θ **no** es lo mismo que θ y después ψ. Un orden distinto da una orientación distinta.

Qué es un FDM

Flight **D**ynamics **M**odel: modelo que representa la física de vuelo de una aeronave. Resuelve un sistema de ecuaciones con la dinámica de cuerpo rígido de una aeronave que evoluciona en el tiempo.

$$\sum \vec{F} = m\,\vec{a} \qquad\qquad \sum \vec{M} = \mathsf{I}\,\dot{\vec{\omega}}$$

Por qué hay que resolverlo numéricamente

Las fuerzas y los momentos de origen aerodinámico **dependen del propio estado** —de la velocidad y de la actitud del instante—, que es justamente lo que se quiere calcular. El sistema es un conjunto de ecuaciones diferenciales no lineales acopladas: **no tiene solución cerrada** y se resuelve con un esquema numérico que avance en el tiempo usando pasos pequeños.

Condiciones iniciales

El FDM requiere un condición inicial de las variables de estado para poder iniciar la integración. Esto se denomina condición de **trim** —vuelo equilibrado— que fija velocidad, actitud y una deflexión de las superficies de control. Tiene que representar una condición *posible* de vuelo.

Qué hace el FDM en cada paso

Qué se resuelve en cada paso de tiempo

1. Con la velocidad y la actitud *actuales* y las posiciones de comando, se calculan las **fuerzas y los momentos** aerodinámicos, propulsivos y de otros orígenes.
2. Dividiendo por masa e inercias se obtienen las **aceleraciones** lineales y angulares.
3. **Integración**: calcula las velocidades usando las aceleraciones y la posición y actitud usando las velocidades.
4. Guarda el **estado nuevo**, ya integrado, para el próximo paso.

$$\underbrace{\vec{x}(t+\Delta t)}_{\text{estado nuevo}} = \underbrace{\vec{x}(t)}_{\text{estado anterior}} + \underbrace{\dot{\vec{x}}(t)}_{\text{sale del paso 2}} \cdot \Delta t$$

Entradas y salidas del FDM

Requiere el **paso de tiempo** para integrar (Δt) y las posiciones de los **comandos**.
Se encarga de integrar, por lo que devuelve la **posición** y la **actitud** del avión.
Hay que **llamarlo una vez por cada paso** que se quiera avanzar (actualizando los comandos cada vez).

La interfaz con el FDM

```
struct ControlInputs {          // ENTRADAS: las posiciones de comando
    float throttle;             // [0, 1]  potencia
    float elevator;             // [rad]   elevador    -> cabeceo
    float aileron;              // [rad]   alerones    -> alabeo
    float rudder;               // [rad]   timon       -> guinada
};
struct AircraftState {          // SALIDAS: en sistemas de referencia AERONAUTICOS
    glm::vec3 position;         // [m]     x=norte  y=este  z=abajo  (NED)
    glm::vec3 body_velocity;    // [m/s]   en ejes cuerpo
    glm::vec3 body_omega;       // [rad/s] en ejes cuerpo
    float phi, theta, psi;      // [rad]   actitud
};

ControlInputs mandos;
...
fdm.update(mandos);             // avanza UN paso de tiempo dt
const AircraftState& x = fdm.getState();
```

Por qué el paso debe ser pequeño y siempre igual

El paso de integración no es un parámetro libre: el modelo funciona para un rango de valores y fuera de este deja de representar la dinámica del avión.

Pequeño

El error de cada paso crece con Δt y los pasos se suman. Con pasos grandes el resultado no sólo se aparta: los modos rápidos de la aeronave —alabeo, cabeceo— se vuelven **inestables** y la simulación diverge.

El FDM que se provee usa $\Delta t = 1/120$ s por defecto.

Siempre el mismo

- Para la misma entrada da **la misma trayectoria** corrida a corrida y máquina a máquina.
- Si el paso lo fijara la máquina el paso más grande se daría justo cuando la máquina está más cargada.

Esto define el problema del game loop

El paso del FDM tiene que ser **pequeño** y **siempre igual**. La generación de los cuadros dependen de la carga de la máquina.

¿Cómo se sincronizan los datos de vuelo con los cuadros del entorno gráfico?

Sincronizar la simulación con el dibujo

El FDM avanza con su paso de tiempo. La escena se dibuja al ritmo de la máquina.

En máquinas diferentes la escena se mueve con **distintas velocidades**. ¿Por qué?

Hay dos relojes en el mismo lazo

- Tiempo de la **máquina** (real): cuánto tarda en dibujarse un cuadro. Depende de la placa, de la ventana y de los procesos que se estén ejecutando en el sistema operativo.
- Tiempo del **modelo físico**: cada cuánto avanza la simulación de la dinámica del avión. Está fijo en el FDM.

Ambos tienen que estar **sincronizados** y el *game loop* es el lugar donde se coordina esta acción.

Por qué la velocidad es diferente en cada máquina

```
while (!glfwWindowShouldClose(window)) {
    glfwPollEvents();
    angulo += 0.01f;            // <-- [rad] incremento POR CUADRO
    render();
    glfwSwapBuffers(window);
}
```

Resultado en dos máquinas distintas

A 60 cuadros por segundo: **0,60 rad/s**.
A 144 cuadros por segundo: **1,44 rad/s**.

El código no cambió. Cambió la máquina.

Solución independiente de la máquina

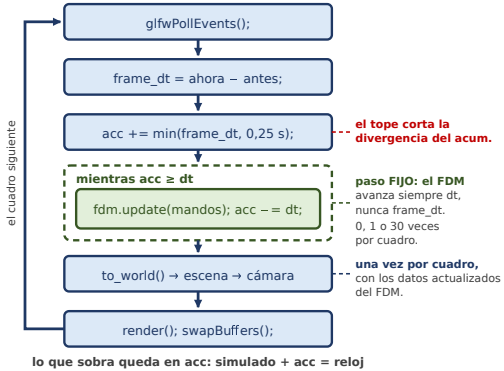
`angulo += velocidad * frame_dt;`

En dos máquinas distintas el movimiento de la escena mantiene la coherencia. Con más de 30 cuadros/seg, no se perciben diferencias en el movimiento.

La plantilla llama `glfwSwapInterval(1)`: los cuadros por segundo son iguales al **refresco del monitor** y por eso pueden cambiar de una máquina a otra (además de la carga).

Estrategia: paso fijo con acumulador

El FDM usa un paso **pequeño e igual siempre**; los cuadros dependen de la capacidad de la máquina.
Una forma de vincularlos es acumular el tiempo transcurrido y luego consumirlo a pasos discretos.




```
acc += std::min(frame_dt, 0.25);

while (acc >= dt) {
    fdm.update(mandos);
    acc -= dt;
}
```

Tener en cuenta

- El resto de tiempo sobrante se acumula para el cuadro siguiente.
- El fdm corre **0, 1 o muchas** veces por cuadro.
- El tope de **0,25 s** corta la divergencia del acumulador: a partir de ahí el loop **deja de simular en tiempo real a propósito** antes que congelarse.

Actividad aúlica: la tabla resuelta

Δt del FDM = 8,33 ms (120 Hz)

cuadro	`frame_dt`	antes	pasos	después
1	16,7 ms	0,00	**2**	0,04
2	33,4 ms	0,04	**4**	0,12
3	250 ms	0,12	**30**	0,22
4	8,0 ms	0,22	**0**	8,22
5	8,0 ms	8,22	**1**	7,89

El invariante

37 pasos $\times$ 8,33 = **308,21 ms** simulados.
En el reloj pasaron **316,10 ms**.

La diferencia es **exactamente** lo que quedó en el acumulador: **7,89 ms**.

$$\text{simulado} + \text{acc} = \text{reloj}$$

La divergencia del acumulador

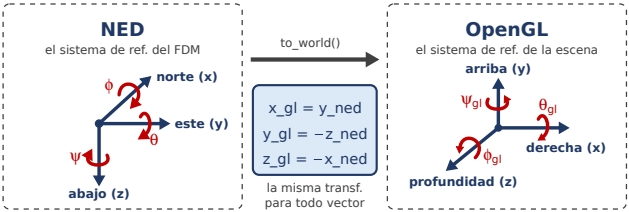
250 ms de cuadro son **30 pasos** de golpe. Si eso alarga el cuadro siguiente, el siguiente pide más todavía. El tope **deja de simular en tiempo real a propósito** antes que congelarse.

Si el acumulador se pusiera en cero

Se pierden **16,22 ms en cinco cuadros**, un **5 %**. Y no se recupera nunca: el reloj de la simulación se atrasa para siempre.

Ajustar sistemas de referencia

El FDM entrega la posición en **NED** y la actitud como tres ángulos definidos sobre ese mismo marco mientras la escena vive en el sistema de **OpenGL**. Ninguno de los dos está mal y ninguno se puede cambiar: cada uno es el estándar de su disciplina.



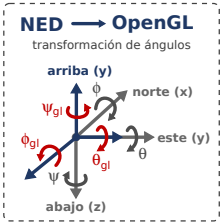
A qué se aplica la transformación

Se convierte **lo que sale del FDM**: posición y actitud de la aeronave para ubicarla en la escena respetando la física del vuelo. Los **ejes cuerpo** no entran en la transformación —la actitud *es* la rotación que lleva de NED a ejes cuerpo—. El terreno, la cámara y el resto de la escena ya están en coordenadas de escena: **no se modifican**.

Como se transforman los ángulos

Si el sistema sigue siendo dextrógiro, girar α alrededor de un eje es visto desde el otro sistema como girar **el mismo** α alrededor del eje transformado.

ángulo	en NED	el mismo eje en GL	se escribe
ψ guiñada	abajo $(0, 0, 1)$	$(0, -1, 0) = -y$	`rotate(-psi, y)`
θ cabeceo	este $(0, 1, 0)$	$(1, 0, 0) = +x$	`rotate(+theta, x)`
ϕ alabeo	norte $(1, 0, 0)$	$(0, 0, -1) = -z$	`rotate(-phi, z)`



El signo de las rotaciones

No son un signo físico. Son la reescritura de *"girar α alrededor de $-y$"* como *"girar $-\alpha$ alrededor de $+y$"* para llamar a las funciones de rotación con ejes positivos. Es notación.

El **orden** de composición se hereda: sigue siendo $\psi \to \theta \to \phi$ para mantener la actitud correcta.

Como es la conversión en código

```
// Unificar esta transformacion en un solo lugar para evitar errores
FlightData to_world(const AircraftState& ned)
{
    FlightData gl;
    gl.position.x =  ned.position.y;   // x_gl =  e
    gl.position.y = -ned.position.z;   // y_gl = -d
    gl.position.z = -ned.position.x;   // z_gl = -n
    gl.phi   = -ned.phi;               // el menos es NOTACION (ver tabla)
    gl.theta =  ned.theta;
    gl.psi   = -ned.psi;
    return gl;
}
// La actitud se arma con ejes positivos, en el orden 3-2-1:
m = glm::rotate(m, gl.psi,   glm::vec3(0.0f, 1.0f, 0.0f));   // 3: guinada
m = glm::rotate(m, gl.theta, glm::vec3(1.0f, 0.0f, 0.0f));   // 2: cabeceo
m = glm::rotate(m, gl.phi,   glm::vec3(0.0f, 0.0f, 1.0f));   // 1: alabeo
```

Estado o evento: el flanco

Para usar el mouse como control de la cámara se propuso la estrategia de *polling de estados*.

Se agrega ahora una nueva estrategia para capturar eventos de entrada del usuario: *callback*.

acción	**naturaleza**	**mecanismo**	**ejemplos**
Control continuo	estado	*polling*	alerones, timón, potencia, mover cámara
Comando discreto	evento	*callback*	pausar, alternar wireframe, cambiar de vista

Polling contesta **"¿está apretada ahora?"**. Un *toggle* necesita **"¿se acaba de apretar?"**, y escrito con *polling* alterna 60 veces por segundo.

Qué es un flanco

La transición entre dos estados es un **flanco**. Una forma de leerlo es esperar un Δt antes de aceptar otro cambio. Sin embargo falla en dos casos: dos pulsaciones rápidas se pierden y una pulsación sostenida sigue cambiando, sólo que más lento. El *callback* de GLFW ya entrega el flanco: distingue `GLFW_PRESS` de `GLFW_REPEAT`.

Cómo asignar las entradas de usuario a comandos del FDM

Los tres controles primarios del avión son ángulos **con signo** y el signo decide para qué lado se mueve el avión. Los ángulos son los que hay que vincular con las entradas del usuario.

mando	movimiento del comando	respuesta de la aeronave	deflexión
Elevador	se empuja el bastón	baja la nariz	$\delta_e > 0$
Alerones	bastón a la derecha	ala derecha abajo	$\delta_a > 0$
Timón	pedal derecho	la nariz va a la derecha	$\delta_r < 0$

Algo más que hay que decidir

¿Qué significa la flecha arriba del teclado? **Si es el bastón** —entonces la nariz baja, que es lo que hace un avión— o **si es la nariz** —y entonces sube, que es lo que espera quien no voló—. Ninguna de las dos es más correcta.

Hay que elegir una convención que sea coherente con lo que espera el usuario del simulador y la realidad que se está simulando.

La proporcionalidad de los comandos y como abordar el problema

Hay un problema al usar el teclado como sistema de entrada para el control de la aeronave: una tecla es discreta —1 o 0— en cambio los comandos de la aeronave son proporcionales ($\delta = k \cdot s$).

Solución óptima: usar un joystick **proporcional** (palanca).

Solución alternativa: emular el comportamiento proporcional por software cuando las entradas son discretas.

mapeo	forma matemática	estado	comportamiento
Directo	$\delta(t) = f(\text{teclas}(t))$	sin memoria	discreto
Rampa	$\delta(t) = \delta(t_0) + \int r \cdot u(\text{teclas})\, dt$	con memoria	proporcional

```
// La rampa, adentro del update() del manejador de entrada:
if (arriba) elevator += rate * frame_dt;    // rate en [rad/s]
if (abajo)  elevator -= rate * frame_dt;
elevator = std::clamp(elevator, min_elevator, max_elevator);
```

Es el mismo esquema numérico de Euler del FDM pero usando el `dt` del cuadro.

La condición inicial del integrador

La rampa para *emular* el comportamiento del movimiento del comando mediante teclado requiere de una condición inicial para empezar a integrar.

Problema: el avión que se descompensa en el primer cuadro

El FDM arranca desde una condición **trim** —equilibrio— y este equilibrio incluye una deflexión de elevador de un **valor específico**. `InputHandler` arranca en **cero**: la tecla no se presionó todavía.

Hay dos lugares guardando la posición de los mandos y sólo uno se inicializó. En el primer cuadro se envía al FDM un comando de elevador que no es el del equilibrio y el avión se descompensa solo sin ninguna razón externa.

Solución

Un método `set_controls()` en `InputHandler` que se llama **antes del loop** junto con el estado de trim. **Estado y comandos de trim son un solo dato repartido en dos lugares:** cambiar uno sin el otro rompe el equilibrio sin causa aparente.

`InputHandler`: ampliación del módulo

```
class InputHandler {
public:
    void update(GLFWwindow* w, float dt);        // polling, 1 vez por cuadro
    const ControlInputs& controls() const;       // lo que consume el FDM
    void set_controls(const ControlInputs& c);   // la condicion inicial
    CameraCommand camera_cmd() const;            // el mouse, del practico anterior
};
```

Qué hace

Traduce teclas y mouse a **números normalizados**, integra las rampas con el `dt` del cuadro y guarda la posición actual de los cuatro comandos de la aeronave.

Qué **no** hace

No integra física, no dibuja, no conoce a `CameraSystem` y **no sabe qué es un alerón**: entrega cuatro números y un comando de cámara.

Práctico 06 — Armar el game loop capturando entradas de usuario

1. **El game loop con paso fijo**: medir `frame_dt`, acumular con tope y consumir de a pasos enteros.
2. **El teclado en** `InputHandler`: los cuatro ejes por *polling* y al menos un comando discreto por *callback*.
3. **Integrar el FDM** y llevar su salida a la escena con `to_world()`.

El punto 3 no bloquea seguir avanzando

Si no se puede poner en marcha el FDM, el proyecto puede continuar: el terreno, el cielo y las cámaras no lo necesitan. Lo que sí conviene tener andando es el **punto 1** porque es la base para incorporar el FDM y el **punto 2** porque permitirá moverse por la escena a pesar de no tener el avión volando.

El avión ya se mueve.

Falta el mundo por el cuál moverse.

Si propongo una grilla de 200×200 celdas,
son **80.000 triángulos** y **40.401 vértices**.

¿Cómo se construye? ¿Se escriben a mano?